

Arquitectura RealDB / P2L — Versión 1

Resumen de cómo está construido el sistema en su estado actual, estable y coherente.

1. Visión general

- **Frontend:** SPA Vue 3 (Vite) con Pinia, Vue Router y Vuetify. Autenticación con Google; dos perfiles: **Empresa (admin)** y **Técnico (technician)**.
 - **Backend:** Servidor Node.js (Express + [Socket.IO](#)). No persiste datos; coordina sesiones, chat y presencia en memoria (protocolo P2L).
 - **Datos:** Local-first. El frontend guarda usuarios, propuestas, daños, empresas y mensajes de chat en **localStorage** (RealDB). El backend solo orquesta en tiempo real.
-

2. Estructura del proyecto

realDB-20251204T204506Z-3-001/

```
└─ src/                # Frontend Vue
|
| └─ api/index.js      # Adaptador API → RealDB + Socket
|
| └─ components/       # ChatModal, Chat, PushNotifications*
|
| └─ composables/useMap.js # Lógica de mapa (Leaflet)
|
| └─ main.js           # Entrada, Pinia, Router, Vuetify, SW
|
| └─ models/index.js   # User, Proposal, Damage, Company, Session, Chat
|
| └─ plugins/vuetify.js
|
| └─ router/index.js, role.js # Rutas y guards por perfil
|
| └─ socket/socketManager.js # Cliente Socket.IO, eventos, presencia
|
| └─ storage/realDB.js  # CRUD sobre localStorage
|
| └─ stores/           # Pinia: auth, presence, notification, map
|
| └─ utils/pushNotifications.js
```

```
| └─ views/          # Login, AdminProfile, TechnicianProfile, ...
|
| └─ modelos_back/
|
|   └─ server.js      # Express + Socket.IO, lógica P2L
|
|   └─ ...
|
| └─ public/sw.js     # Service Worker (push)
|
└─ Documentación arquitectura/
```

3. Capas del frontend

3.1 Entrada y enrutado

- **main.js:** Crea la app Vue, Pinia, Router, Vuetify; registra el Service Worker; espera Phosphor Icons; llama a `useAuthStore().checkAuthStatus()` antes de montar.
- **Router:** Tres rutas: `/` (Login), `/admin` (AdminProfile), `/technician` (TechnicianProfile). `beforeEach` usa `useAuthStore()` para:
 - Redirigir a login si la ruta requiere auth y no hay sesión.
 - Redirigir a admin o technician si ya está logueado y entra a login.
 - Comprobar que el perfil (company vs technician) coincida con la ruta; si no, redirige al perfil correcto.

3.2 Autenticación (`useAuthStore`)

- Login con Google: se recibe `userData` y `profile` (company | technician). Se crea/actualiza usuario en RealDB vía `api.createUser`, se guarda token y datos en `localStorage`, se normaliza usuario (id, email, Email, etc.) y se expone en el store.
- `checkAuthStatus`: lee token y `user_data` de `localStorage`, verifica expiración y existencia del usuario en RealDB (`api.getUserByEmail`); si todo es válido, rehidrata el store.
- El socket **no** se conecta en el store; la conexión se hace cuando el perfil (Admin o Technician) necesita Socket (tras login efectivo).

3.3 API (api/index.js)

- **APIAdapter** simula REST sobre **localStorage**: get/post/put/delete por colección (users, proposals, damages, companies).
- Usa **realDB** para CRUD y **modelos** (UserModel, ProposalModel, etc.) para crear/actualizar con forma fija.
- Al crear/actualizar propuestas o daños, además escribe en RealDB y notifica por socket (socketManager.notifyNewProposal, notifyNewDamage, send('proposalUpdate'|'damageUpdate')).
- Métodos
específicos: getUserByEmail, getNearbyUsers, getUserLocation (estos últimos delegan al socketManager).

3.4 Almacenamiento (storage/realDB.js)

- Una clave realDB en localStorage con un único objeto: { user, proposals, damages, companies, technicians, sessions, lastSync }.
- Operaciones: create, read, update, delete por colección; generateId para IDs únicos; sin backend de base de datos.

3.5 Modelos (models/index.js)

- **UserModel, ProposalModel, DamageModel, CompanyModel:** Crean/actualizan entidades con campos fijos y las guardan en RealDB.
- **SessionModel:** Sesiones de socket (socketId, userId, profile, location, isOnline); usado desde el front para reflejar sesión en RealDB.
- **ChatModel:** Mensajes **no** van en el objeto realDB; se guardan en claves chat_<userId>_<chatId> en localStorage.
Métodos: createMessage, getMessagesByChatId, saveMessages, getChatKey, getAllChatIds, etc. Aísla mensajes por usuario.

3.6 Socket (socket/socketManager.js)

- **Singleton** (export default instancia). Conecta con [Socket.IO](https://socket.io/) al backend usando userData.id (obligatorio), profile, location, email, pushSubscription.
- **Path:** En producción /p2l/socket.io, en desarrollo /socket.io.

- **Eventos que recibe del servidor:** connect/disconnect, userNearby, proposalUpdate, damageUpdate, userStatusChange, chatMessage, chatUserJoined, userStatusResponse, presence:snapshot, presence:chat_opened, presence:chat_closed, presence:app_disconnected, notification:new_message, chat:sent, chat:delivered, chat:received, chat:read.
- **Eventos que emite al servidor:** requestNearbyUsers, statusChange, presenceUpdate, joinChat, leaveChat, sendChatMessage/chatMessage, chat:read, checkUserStatus, etc.
- Reenvío interno: al recibir un evento del socket, actualiza estado interno (p. ej. nearbyUsers) y llama a this.emit(evento, data) para suscriptores registrados con on().
- **Presencia:** Los eventos presence:* se delegan al **usePresenceStore** (handlePresenceSnapshot, handlePresenceChatOpened, etc.). Además detecta visibilidad de pestaña (Page Visibility / focus-blur) y envía presenceUpdate con isIdle.
- **Geolocalización:** getUserLocation() (navigator.geolocation); se envía en auth al conectar.
- Tras conectar, crea una sesión en RealDB vía SessionModel.

3.7 Stores Pinia

Store	Rol
useAuthStore	loggedIn, user, profile, token; login (Google + RealDB), logout, checkAuthStatus.
usePresenceStore	presenceByChat (Map por chatId). Solo reacciona a eventos del backend: presence:snapshot, presence:chat_opened, presence:chat_closed, presence:app_disconnected. Fuente de verdad de presencia es el backend.

Store	Rol
useNotificationStore	notifications (array); showNotification, removeNotification, clearNotifications. Usado por App.vue y por notificaciones de mensajes.
useMapStore	userLocation, damages, markers, nearbyUsers; setters y helpers para mapa.

3.8 Vistas principales

- **Login.vue:** Selector de perfil (Empresa / Técnico), Google Sign-In, enlaces a Condiciones de uso y Términos de privacidad. Tras login exitoso llama a `authStore.login({ userData, profile })` y el router redirige a `admin` o `technician`.
- **AdminProfile.vue:** Perfil Empresa. Mapa, solicitudes (daños), propuestas recibidas, “Conversa con usuarios cercanos”, ChatModal. Escucha `openChatFromNotification` (CustomEvent) para abrir chat desde notificación; usa `selectedOtherUserForChat` y propuestas para el contexto del chat.
- **TechnicianProfile.vue:** Perfil Técnico. Mapa, oportunidades cercanas (daños), propuestas enviadas, “Conversa con usuarios cercanos”, ChatModal. Misma mecánica de `openChatFromNotification` y chat.

En ambos perfiles, el socket se conecta cuando hace falta (tras tener usuario con id); el chat se abre por propuesta (con `proposalId`) o por usuario cercano (chat directo, sin propuesta).

3.9 Chat y presencia

- **ChatModal.vue:** Un solo componente para ambos flujos: (1) chat ligado a propuesta, (2) chat directo con usuario cercano. Recibe `proposal/otherUser`; si no hay `proposal`, `chatId` puede ser un id de “room” directo. Al abrir hace `joinChat`; al cerrar hace `leaveChat`. Lee presencia del otro usuario desde **usePresenceStore** (derivada de `presence:snapshot`, `presence:chat_opened`, `presence:chat_closed`). Los mensajes se cargan/guardan con **ChatModel** (`localStorage` por usuario y `chatId`) y se envían/reciben por socket.

- **Presencia:** El backend es la única fuente de verdad. El front no calcula “online/offline”; solo aplica los handlers del presenceStore a los eventos presence:*. Estados: OFFLINE, ONLINE, IN_CHAT (ver [PATRON-PRESENCIA-CHAT.md](#)).

3.10 App.vue

- Renderiza <router-view />, lista de notificaciones globales (notificationStore) y PushNotificationsModal.
- Escucha notification:new_message por socketManager; al recibir mensaje guarda en ChatModel, muestra notificación con acción “Abrir Chat” que dispara el CustomEvent openChatFromNotification (detail: proposalId, chatId) para que AdminProfile o TechnicianProfile abran el ChatModal.

4. Backend (modelos_back/server.js)

- **Express:** CORS, JSON, sin rutas REST de negocio; solo sirve para levantar el servidor HTTP.
- **Socket.IO:** path /socket.io, middleware authenticateSocket: exige userId y profile en handshake; opcionales: location, email, whatsapp, pushSubscription. Guarda en socket: userId, profile, location, email, geohash (por ubicación).
- **Estado en memoria (sin persistencia):**
 - activeSessions (Map socketId → sesión)
 - userLocations (Map userId → location)
 - geohashRooms (geohash → Set socketIds)
 - chatRooms (roomId → Set socketIds)
 - offlineMessageQueue (por usuario, para cola offline)
 - pushSubscriptions (por userId, para web push)
- **Conexión:** Al conectar se detectan duplicados (mismo userId/email y perfil); se puede cerrar la sesión anterior. Se registra la sesión con location, email, profile, isOnline, isIdle, activeChatId (null al conectar). Se emite userStatusChange global.

- **Eventos principales que escucha:**
 - requestNearbyUsers: calcula usuarios en el mismo geohash (o cercanos), responde con userNearby.
 - statusChange / presenceUpdate: actualiza isIdle y/o activeChatId en sesión.
 - joinChat: payload con chatId, proposalId, otherUserId, etc. Añade socket a chatRooms, actualiza activeChatId en sesión, emite presence:chat_opened al otro usuario, presence:snapshot al que entró, y opcionalmente chatUserJoined.
 - leaveChat: quita de chatRooms, limpia activeChatId, emite presence:chat_closed (y chatUserLeft) al otro.
 - sendChatMessage / chatMessage: enruta mensaje al destinatario por socket o por room; emite chatMessage, chat:sent, chat:delivered, chat:received; si el destinatario no está en el chat puede enviar notification:new_message y/o encolar para offline.
 - chat:read: reenvío al emisor.
 - newProposal, proposalUpdate, newDamage, damageUpdate, pendingProposals, pendingDamages, etc.: sincronización de propuestas y daños entre admin y técnicos (por email/userId y geohash).
 - disconnect: limpia sesión, notifica presence:chat_closed / presence:app_disconnected a compañeros de chat, emite userStatusChange.
- **Push:** web-push para notificaciones cuando el usuario no está conectado o no está en el chat; suscripciones guardadas por userId en pushSubscriptions.

5. Flujos de datos resumidos

1. **Login:** Google → AuthStore.login → api.createUser (RealDB) → localStorage + Pinia → Router dirige a /admin o /technician.
2. **Conexión socket:** En el perfil se usa socketManager; si hay user con id, connect(userData) con auth (userId, profile, location, email, pushSubscription). Backend valida y registra en activeSessions.

3. **Usuarios cercanos:** Front emite requestNearbyUsers; backend calcula por geohash y responde userNearby; socketManager actualiza nearbyUsers y emite a suscriptores.
4. **Chat (abrir):** Usuario abre ChatModal (desde propuesta o desde lista de cercanos). Front emite joinChat con chatId/proposalId y otherUserId. Backend actualiza chatRooms y activeChatId, emite presence:snapshot al que entra y presence:chat_opened al otro. Front (presenceStore) actualiza presencia; no abre modales por presencia.
5. **Mensajes:** Usuario escribe; front envía por socket (sendChatMessage/chatMessage); backend entrega al otro socket o emite notification:new_message y/o encola. El receptor recibe chatMessage (y opcionalmente notification:new_message); App o ChatModal guardan en ChatModel y muestran notificación con “Abrir Chat” si aplica.
6. **Cerrar chat:** Front emite leaveChat; backend emite presence:chat_closed al otro; presenceStore actualiza estado a ONLINE (no OFFLINE).
7. **Propuestas / daños:** Creación y actualización en front vía API (RealDB + notificaciones socket); backend reenvía proposalUpdate/damageUpdate a los sockets interesados (admin o técnicos por email/geohash).

6. Tecnologías clave

Capa	Tecnologías
Frontend	Vue 3, Vite, Pinia, Vue Router, Vuetify, Socket.IO Client, Phosphor Icons
Backend	Node.js, Express, Socket.IO , CORS, dotenv, web-push
Persistencia	localStorage (RealDB), claves chat_* para mensajes
Infra	Service Worker (public/sw.js) para push; en producción nginx (ej. /p2l/socket.io)

7. Convenciones y decisiones

- **Sin evento chat:open desde backend:** La UI de “abrir chat” se controla solo en el front (botones, notificación “Abrir Chat” → CustomEvent openChatFromNotification). Los eventos presence:* solo actualizan estado de presencia.
- **Un solo ChatModal:** Dos entradas (propuesta vs usuario cercano) comparten el mismo componente; el backend distingue por proposalId (null = chat directo) y roomId/chatId.
- **IDs de usuario:** Se normalizan en auth (id, sub, googleId, email, Email) para evitar fallos entre front y backend.
- **Presencia:** Solo el backend emite presencia; el front solo reacciona (presenceStore). No se usa timeout/nextTick para sincronizar presencia.

Documento: Arquitectura v1 — Estado estable y coherente del sistema RealDB/P2L.